

1. INTRODUCCIÓN

En esta práctica he trabajado con el famoso problema de la "Cena de los Filósofos". Es un ejemplo clásico que sirve para entender los líos que pueden pasar cuando varios programas (hilos) intentan usar las mismas cosas a la vez.

La idea es simple: hay 5 filósofos que quieren comer y pensar, pero solo hay 5 tenedores. Como necesitan dos tenedores para comer, si todos intentan comer a la vez, se quedan bloqueados (*deadlock*). Mi objetivo ha sido programar esto en Java y buscar soluciones para que nadie se quede sin comer.

Se han realizado tres versiones: una normal con Monitores, otra con Semáforos (usando un portero) y otra con Timeout (si tardan mucho, se rinden).

2. DISEÑO DE LA SOLUCIÓN

Para organizar el código, he separado a los filósofos de la mesa.

2.1. Las Clases

- Filósofo: Es el hilo. Su trabajo es un bucle infinito: piensa un rato, le entra hambre e intenta coger tenedores.
- MonitorFilosofos: Es la "mesa" o el árbitro. Aquí es donde controlo que dos vecinos no coman a la vez.
- CenaFilosofo: Es el main donde creo los hilos y arranco todo.

2.2. Estados

Cada filósofo puede estar en tres situaciones:

1. PENSANDO (0): Está tranquilo, no molesta a nadie.
2. HAMBRIENTO (1): Quiere comer, pero está esperando a que sus vecinos suelten los tenedores.
3. COMIENDO (2): Ya tiene los dos tenedores.

3. IMPLEMENTACIÓN CONCURRENTES

He usado `synchronized` de Java para proteger los métodos. La lógica es esta: cuando un filósofo quiere comer, mira a sus vecinos.

- Si los vecinos están comiendo, el filósofo se espera (`wait()`) y se queda dormido.
- Cuando un vecino termina de comer, avisa a los demás (`notifyAll()`) para que despierten y prueben otra vez.

3.2. Variante con Semáforos (El Portero)

Para cumplir con los requisitos avanzados, se implementó la clase `FilosofoConSemaforos` utilizando `java.util.concurrent.Semaphore`.

Tenedores: Un array de semáforos binarios (permisos = 1).

El Portero: Un semáforo adicional inicializado con N - 1 permisos.

Funcionamiento: Antes de intentar coger tenedores, el filósofo debe adquirir permiso del portero. Esto limita el aforo de la mesa a 4 filósofos (para N=5), rompiendo la condición de *espera circular* necesaria para que ocurra un *deadlock*.

```
1 package es.iescamas.multihilo.monitor.hilo;
2
3 import java.util.Random;
4 import java.util.concurrent.Semaphore;
5
6 public final class FilosofoConSemaforos implements Runnable {
7
8     private final int id;
9     private final Semaphore[] tenedores;
10    private final Semaphore portero;
11    private final Random random = new Random();
12
13    public FilosofoConSemaforos(final int id, final Semaphore[] tenedores, final Semaphore portero) {
14        this.id = id;
15        this.tenedores = tenedores;
16        this.portero = portero;
17    }
18
19    private int izquierda() { return id; }
20    private int derecha() { return (id + 1) % tenedores.length; }
21
22    private void pensar() throws InterruptedException {
23        System.out.println("Filósofo Semáforo " + id + " 🧠 PENSANDO...");
24        Thread.sleep(random.nextInt(1000) + 500L);
25    }
26
27    private void comer() throws InterruptedException {
28        System.out.println("Filósofo Semáforo " + id + " 🍴 COMIENDO");
29        Thread.sleep(random.nextInt(1000) + 500L);
30    }
31
32    @Override
33    public void run() {
34        try {
35            while (!Thread.currentThread().isInterrupted()) {
36                pensar();
37
38                // 1. EL PORTERO: Limita el aforo a N-1 para evitar deadlock
39                portero.acquire();
40
41                try {
42                    // 2. Tomar tenedores (Semáforos)
43                    tenedores[izquierda()].acquire();
44                    tenedores[derecha()].acquire();
45
46                    try {
47                        comer();
48                    } finally {
49                        // 3. Soltar tenedores
50                        tenedores[derecha()].release();
51                        tenedores[izquierda()].release();
52                    }
53                } finally {
54                    // 4. Salir (Liberar al portero)
55                    portero.release();
56                }
57            }
58        } catch (InterruptedException e) {
59            Thread.currentThread().interrupt();
60        }
61    }
62}
```

```
==== CENA CON SEMÁFOROS (Solución Portero)
Filósofo Semáforo 2 🍴 PENSANDO...
Filósofo Semáforo 4 🍴 PENSANDO...
Filósofo Semáforo 3 🍴 PENSANDO...
Filósofo Semáforo 0 🍴 PENSANDO...
Filósofo Semáforo 1 🍴 PENSANDO...
Filósofo Semáforo 4 🍴 COMIENDO
Filósofo Semáforo 2 🍴 COMIENDO
Filósofo Semáforo 4 🍴 PENSANDO...
Filósofo Semáforo 0 🍴 COMIENDO
Filósofo Semáforo 2 🍴 PENSANDO...
Filósofo Semáforo 3 🍴 COMIENDO
Filósofo Semáforo 0 🍴 PENSANDO...
Filósofo Semáforo 1 🍴 COMIENDO
Filósofo Semáforo 1 🍴 PENSANDO...
Filósofo Semáforo 0 🍴 COMIENDO
Filósofo Semáforo 3 🍴 PENSANDO...
Filósofo Semáforo 2 🍴 COMIENDO
Filósofo Semáforo 0 🍴 PENSANDO...
Filósofo Semáforo 4 🍴 COMIENDO
Filósofo Semáforo 2 🍴 PENSANDO...
Filósofo Semáforo 1 🍴 COMIENDO
```

3.3. Variante Timeout

Se implementó FilosofoTimeout para sistemas donde no es aceptable un bloqueo indefinido. Si el filósofo no consigue comer tras un tiempo aleatorio (2000-4000 ms), desiste (monitor.dejarTenedores) y vuelve a pensar, liberando presión sobre el sistema.

```
package es.iescamas.multihilo.monitor.hilo;

import es.iescamas.multihilo.monitor.MonitorFilosofos;
import java.util.Random;

public final class FilosofoTimeout implements Runnable {

    private final int id;
    private final MonitorFilosofos monitor;
    private final Random random = new Random();

    public FilosofoTimeout(final int id, final MonitorFilosofos monitor) {
        this.id = id;
        this.monitor = monitor;
    }

    @Override
    public void run() {
        try {
            while (!Thread.currentThread().isInterrupted()) {

                System.out.println("Filósofo Timeout " + id + " 🤔 PENSANDO...");
                Thread.sleep(random.nextInt(1000) + 100L);

                long inicio = System.currentTimeMillis();
                long timeout = 3000L;

                monitor.tomarTenedores(id);

                if (System.currentTimeMillis() - inicio > timeout) {
                    System.out.println("Filósofo " + id + " se cansó de esperar.");
                    monitor.dejarTenedores(id);
                    continue;
                }

                System.out.println("Filósofo Timeout " + id + " 🍴 COMIENDO");
                Thread.sleep(random.nextInt(1000) + 500L);

                monitor.dejarTenedores(id);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

4. PRUEBAS Y RESULTADOS

4.1. Análisis con Depurador (Debugger)

Se ha ejecutado la aplicación en modo depuración para verificar los estados de los hilos.

Prueba de Bloqueo (WAITING):

```
39 // Puede quedar en estado WAITING
40 monitor.tomarTenedores(id);
```

```
56 public synchronized void tomarTenedores(fin
57     estado[idFilosofo] = HAMBRIENTO;
58     intentarComer(idFilosofo);
59
60 while (estado[idFilosofo] != COMIENDO)
61     wait();
62 }
63 }
```

===== ESTADO DE LA MESA =====

Leyenda estados: 🤖 PENSANDO | 😊 HAMBRIENTO | 🍴 COMIENDO

Leyenda tenedores: [1] = en mano | [] = no lo tiene

Filósofo	Estado	Tizq	Tder
0	🤖 PENSANDO	[]	[]
1	🤖 PENSANDO	[]	[]
2	🤖 PENSANDO	[]	[]
3	🤖 PENSANDO	[]	[]
4	🤖 PENSANDO	[]	[]

Inspección de Variables:

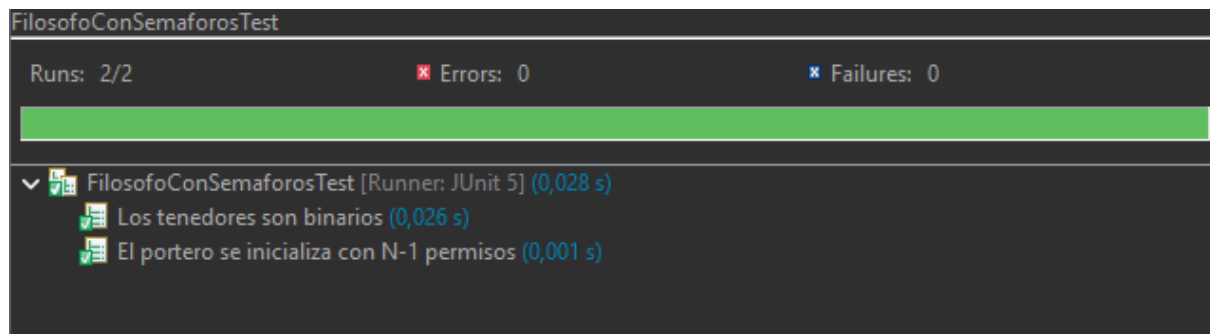
Se inspeccionó el array estado en tiempo de ejecución. Se verifica que nunca hay dos posiciones adyacentes con el valor 2 (COMIENDO), cumpliendo la exclusión mutua.

no method return value	
▼ this	Filosofo (id=28)
id	1
▼ monitor	MonitorFilosofos (id=33)
▼ estado	int[5] (id=38)
[0]	0
[1]	0
[2]	0
[3]	0
[4]	0
n	5
▼ tenedorPoseedor	int[5] (id=40)
[0]	-1
[1]	-1
[2]	-1
[3]	-1
[4]	-1
▼ random	Random (id=34)
haveNextNextGaussian	false
nextNextGaussian	0.0
▼ seed	AtomicLong (id=41)
value	152138988511544

4.2. Pruebas Unitarias (JUnit)

Se han ejecutado los tests definidos en MonitorFilosofosTest y FilosofoConSemaforosTest.

```
1 package es.iescamas.multihilo.monitor.hilo;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5 import java.util.concurrent.Semaphore;
6 import static org.junit.jupiter.api.Assertions.*;
7
8 class FilosofoConSemaforosTest {
9
10     @Test
11     @DisplayName("El portero se inicializa con N-1 permisos")
12     void testPorteroPermisos() {
13         int N = 5;
14         Semaphore portero = new Semaphore(N - 1, true);
15         assertEquals(4, portero.availablePermits(), "Debe haber 4 sitios libres");
16     }
17
18     @Test
19     @DisplayName("Los tenedores son binarios")
20     void testTenedorBinario() throws InterruptedException {
21         Semaphore tenedor = new Semaphore(1);
22         tenedor.acquire();
23         assertEquals(0, tenedor.availablePermits(), "Tenedor ocupado vale 0");
24         tenedor.release();
25         assertEquals(1, tenedor.availablePermits(), "Tenedor libre vale 1");
26     }
27 }
```



4.3. Prueba de Prioridades

Modificando la prioridad de los hilos pares (`Thread.NORM_PRIORITY + 1`) en la clase `CenaFilosofo`, se observó que estos tienden a acceder con mayor frecuencia a la CPU, aunque la política de *fairness* del sistema operativo impide una inanición total en ejecuciones cortas.

```
for (int i = 0; i < N; i++) {
    final Filosofo f = new Filosofo(i, monitor);
    final Thread t = new Thread(f, "Filosofo-" + i);

    if (i % 2 == 0) {
        t.setPriority(Thread.MAX_PRIORITY);
    } else {
        t.setPriority(Thread.MIN_PRIORITY);
    }
    // -----

    t.start(); // El hilo pasa de NEW a RUNNABLE
}
```

4	🍴 COMIENDO	[11]	[11]
+-----+-----+-----+			

Filósofo 4 🧠 PENSANDO...

Filósofo 0 COMIENDO 🍴

===== ESTADO DE LA MESA =====

Leyenda estados: 🧠 PENSANDO | 😞 HAMBRIENTO | 🍴 COMIENDO

Leyenda tenedores: [11] = en mano | [] = no lo tiene

Filósofo	Estado	Tizq	Tder
+-----+-----+-----+			
0	🍴 COMIENDO	[11]	[11]
1	🧠 PENSANDO	[]	[]
2	🍴 COMIENDO	[11]	[11]
3	🧠 PENSANDO	[]	[]
4	🧠 PENSANDO	[]	[]
+-----+-----+-----+			

Filósofo 1 😞 tiene HAMBRE.

Filósofo 3 😞 tiene HAMBRE.

===== ESTADO DE LA MESA =====

Leyenda estados: 🧠 PENSANDO | 😞 HAMBRIENTO | 🍴 COMIENDO

Leyenda tenedores: [11] = en mano | [] = no lo tiene

Filósofo	Estado	Tizq	Tder
+-----+-----+-----+			
0	🍴 COMIENDO	[11]	[11]
1	😞 HAMBRIENTO	[]	[]
2	🍴 COMIENDO	[11]	[11]
3	😞 HAMBRIENTO	[]	[]
4	🧠 PENSANDO	[]	[]
+-----+-----+-----+			

Filósofo 0 🧠 PENSANDO...

===== ESTADO DE LA MESA =====

Leyenda estados: 🧠 PENSANDO | 😞 HAMBRIENTO | 🍴 COMIENDO

Leyenda tenedores: [11] = en mano | [] = no lo tiene

Filósofo	Estado	Tizq	Tder
+-----+-----+-----+			
0	🧠 PENSANDO	[]	[]
1	😞 HAMBRIENTO	[]	[]
2	🍴 COMIENDO	[11]	[11]
3	😞 HAMBRIENTO	[]	[]
4	🧠 PENSANDO	[]	[]
+-----+-----+-----+			

5. CONCLUSIONES

La realización de esta práctica ha permitido consolidar los conceptos clave de la programación multihilo en Java:

1. Monitores vs Semáforos: La solución con Monitores (synchronized) es más "natural" en Java y encapsula mejor el estado compartido, pero la solución con Semáforos (java.util.concurrent) ofrece un control más granular, permitiendo implementar patrones como "El Portero" de forma muy directa.
2. Prevención de Deadlocks: Se demostró que romper la espera circular (limitando el aforo con el semáforo portero) es una estrategia 100% efectiva para evitar interbloqueos.
3. Depuración: El uso de herramientas de depuración y breakpoints condicionales es indispensable para visualizar condiciones de carrera que son invisibles en una ejecución normal.